



Univerzitet u Beogradu
Matematički fakultet

Seminarski rad iz Verifikacije softvera

Clang semantička provera: Čiste i konstantne funkcije

Profesor i asistent:

prof. dr Milena Vujošević Jančić
Ivan Ristović

Studenti:

Đurđa Milošević 1008/2025
Anja Milutinović 1011/2025

Datum: 2026.

1 Opis problema

Savremeni C i C++ kompajleri podržavaju različite funkcijske atribute kojima programer može eksplicitno da opiše osobine funkcija. Dva atributa ove vrste koja su se izučavala u ovom radu su **pure** i **const**. Oba atributa opisuju funkcije koje nemaju sporedne efekte (eng. *observable side effects*), ali se razlikuju po tome koliko je funkciji dozvoljeno da pristupa memoriji i od čega sme da zavisi njena povratna vrednost.

Ove informacije kompajler koristi za optimizacije zasnovanih na pretpostavci da će funkcija za iste ulazne argumente vratiti isti rezultat. Atributi **pure** i **const** prvenstveno služe kao informacije koje programer prosleđuje kompajleru, kompajler ne proverava da li implementacija zaista zadovoljava uslove koje atribut nameće, već pretpostavlja da je deklaracija tačna i na osnovu nje primenjuje odgovarajuće optimizacije. Zbog toga pogrešno označavanje funkcije može dovesti do neispravnih optimizacija i promjenjivog ponašanja programa. U ovom radu Clang Static Analyzer korišćen je za implementaciju proveravača koji analizira funkcije označene atributima **pure** i **const**. Tokom simboličkog izvršavanja proveravač prati da li funkcija proizvodi sporedne efekte, kao i da li pristupa memoriji na način koji nije dozvoljen za odgovarajući atribut.

1.1 Pure funkcije

Funkcija označena atributom **pure** ne sme da menja stanje programa koje je vidljivo ostatku programa. Jedini rezultat njenog izvršavanja sme biti povratna vrednost funkcije. Funkcija sa atributom **pure** sme da čita proizvoljne nepromenljive (eng. *non-volatile*) objekte iz memorije, uključujući globalne promenljive i objekte do kojih dolazi preko pokazivača ili referenci. Međutim, nije dozvoljeno da ih menja niti da izvršava bilo kakve druge operacije koje proizvode sporedne efekte. Ukoliko se vrednosti koje funkcija čita promene između dva poziva, može se promeniti i rezultat funkcije, pa se optimizacije mogu primeniti samo dok kompajler može da garantuje da se relevantno stanje nije promenilo. [1]

1.2 Const funkcije

Atribut **const** predstavlja strožiju verziju atributa **pure**. Pored toga što funkcija ne sme da proizvodi sporedne efekte, njena povratna vrednost sme da zavisi isključivo od vrednosti njenih argumenata. Odnosno, funkcija ne sme da čita promenljive čija bi promena mogla da utiče na rezultat funkcije. Dozvoljeno je jedino čitanje objekata koji predstavljaju konstante i čija

vrednost ne može da se promeni tokom izvršavanja programa. Zbog toga se ovakav atribut uglavnom ne koristi za funkcije koje primaju pokazivače ili reference, jer sadržaj memorije na koji oni pokazuju može da se promeni između dva poziva funkcije. [1]

Osobina	pure	const
Nema sporednih efekata	✓	✓
Može da čita globalne promenljive	✓	×
Može da menja vrednost globalne promenljive	×	×
Može da čita preko pokazivača/referenci	✓	×
Može da piše preko pokazivača/referenci	×	×

Tabela 1: Poređenje atributa `pure` i `const`

2 Opis arhitekture sistema

Arhitektura sistema organizovana je kroz više komponenti sa jasno razdvojenim odgovornostima. Centralnu komponentu predstavlja klasa `PureFunctionChecker`, koja implementira callback metode Clang Static Analyzer-a. Ove metode se aktiviraju u različitim fazama analize, kao što su ulazak u funkciju, poziv druge funkcije, upis u memoriju i završetak analize funkcije.

`ProgramState` se koristi za čuvanje informacija tokom simboličkog izvršavanja. U fajlu `PureState` definisane su dodatne vrednosti koje checker upisuje u stanje, kao što su trenutna dubina analize, vrsta funkcije na određenoj dubini i do tada otkriveni sporedni efekti.

Komponenta `PureBugReporter` zadužena je za formiranje i prijavljivanje dijagnostičkih poruka. Na taj način je logika detekcije nepravilnosti odvojena od logike prikazivanja upozorenja korisniku.

Pomoćne funkcije za prepoznavanje `pure` i `const` funkcija izdvojene su u komponentu `PurityUtils`. Ove funkcije analiziraju deklaraciju funkcije i proveravaju prisustvo odgovarajućih GNU atributa ili `clang::annotate` anotacija.

Checker podržava tri režima rada: analizu samo `pure` funkcija, samo `const` funkcija ili obe vrste funkcija. Režim rada bira se prilikom pokretanja analizatora, prosleđivanjem odgovarajuće vrednosti opcije `Mode`. Na ovaj način ista implementacija može da se koristi za različite potrebe analize bez uvođenja odvojenih checker-a sa dupliciranom logikom.

2.1 Clang Static Analyzer

Clang Static Analyzer (CSA) predstavlja alat za statičku analizu programa koji je deo LLVM/Clang kompajlerske infrastrukture. Analiza se zasniva na simboličkom izvršavanju (engl. *symbolic execution*), pri čemu se istovremeno razmatraju različiti tokovi izvršavanja i održava apstraktno stanje programa koje opisuje vrednosti promenljivih, sadržaj memorije i druge relevantne informacije.

CSA omogućava proširivanje postojećeg sistema dodavanjem sopstvenih checker-a. Checker predstavlja komponentu koja reaguje na određene događaje tokom analize programa i proverava specifična svojstva izvornog koda. U tu svrhu koristi se sistem callback metoda koje CSA automatski poziva kada tokom simboličkog izvršavanja dođe do određenog događaja.

U implementiranom checker-u koriste se callback metode koje Clang Static Analyzer poziva u različitim fazama simboličkog izvršavanja programa. Svaka callback metoda ima jasno definisanu odgovornost u procesu analize.

Za evidentiranje pronađenih sporednih efekata koristi se bit-mask. Svakoj vrsti sporednog efekta dodeljen je poseban bit, pri čemu se efekti mogu kombinovati primenom bitovske operacije OR. Prilikom detekcije novog sporednog efekta odgovarajući bit postavlja se u maski, dok se pri završetku analize funkcije jednostavnom bitovskom proverom može utvrditi koje vrste narušavanja su zabeležene.

3 Opis rešenja problema

Analiza se sastoji iz četiri osnovna koraka: identifikacije funkcija koje treba proveravati, praćenja trenutnog stanja analize, detekcije sporednih efekata i prijavljivanja dijagnostike po završetku analize funkcije. Jedna od ključnih prednosti implementacije jeste korišćenje path-sensitive analize koju obezbeđuje Clang Static Analyzer. Umesto analize sintaksne strukture programa, proveravač razmatra različite tokove izvršavanja i održava posebno stanje za svaki od njih. Zbog toga se upozorenje prijavljuje samo ukoliko je sporedni efekat dostižan na posmatranom toku izvršavanja.

U okviru `ProgramState` proveravač održava trenutni nivo ugnježđenja analiziranih `pure` funkcija (`PureDepth`), tip funkcije na svakom nivou ugnježđenja i skup detektovanih sporednih efekata za svaki nivo poziva. Na ovaj način moguće je nezavisno analizirati ugnježdene pozive funkcija, a nakon završetka analize propagirati pronađene sporedne efekte ka funkciji koja ih je pozvala.

3.1 Pure checker

Pure checker je proveravač namenjen proveriti funkcija označenih atributom `pure`. Tokom analize proveravaju se sledeći uslovi koje `pure` funkcija mora da zadovolji:

- ne sme da upisuje u globalne promenljive,
- ne sme da upisuje preko pokazivača,
- ne sme da upisuje preko referenci,
- ne sme da poziva funkcije za koje nije moguće proveriti da li su `pure`.

Na primer, ukoliko funkcija poziva drugu funkciju koja menja globalnu promenljivu samo kada je argument negativan, a analizom je utvrđeno da argument na posmatranom putu uvek ima nenegativnu vrednost, put koji sadrži upis u globalnu promenljivu smatra se nedostižnim i upozorenje se neće prijaviti. Sa druge strane, ukoliko takva garancija ne postoji, proveravač prijavljuje da funkcija narušava uslove atributa `pure`.

Listing 1: Path-sensitive analiza

```
1  int globalCounter = 0;
2
3  void update_global_if_negative(int x)
4  {
5      if (x < 0)
6          globalCounter = 10;
7  }
8
9  [[clang::annotate("pure")]]
10 int pure_safe()
11 {
12     unsigned short x = 10;
13     update_global_if_negative(x); // nema upozorenja
14     return x;
15 }
16
17 [[clang::annotate("pure")]]
18 int pure_unsafe(int x)
19 {
20     update_global_if_negative(x); // upozorenje
21     return x;
22 }
```

Metoda `checkBeginFunction()` poziva se prilikom ulaska u funkciju. U ovoj fazi proverava se da li je funkcija označena atributom `pure` ili `const`. Ukoliko jeste, u `ProgramState` se beleže informacije o dubini trenutne analize, vrsti funkcije koja se proverava i inicijalnom stanju bez pronađenih sporrednih efekata.

Metoda `checkBind()` poziva se svaki put kada tokom simboličkog izvršavanja dođe do upisa vrednosti u memoriju. Analizom lokacije upisa utvrđuje se da li se radi o upisu u globalnu promenljivu, preko pokazivača ili preko reference. U slučaju detektovanog narušavanja pravila odgovarajući sporedni efekat upisuje se u trenutno stanje analize.

Metoda `checkPreCall()` poziva se pre poziva druge funkcije. Ako pozivana funkcija ima dostupno telo, CSA nastavlja analizom njene implementacije. U suprotnom, dozvoljenost poziva određuje se na osnovu atributa pozivane funkcije i vrste funkcije pozivaoca. Funkcija označena atributom `const` može pozivati samo `const` funkcije, dok `pure` funkcija može pozivati i `pure` i `const` funkcije.

Metoda `checkEndFunction()` poziva se pri završetku analize funkcije. Ukoliko su tokom izvršavanja zabeleženi sporedni efekti, kreira se odgovarajući dijagnostički izveštaj. Nakon toga informacije vezane za završenu funkciju uklanjaju se iz `ProgramState`-a.

3.2 Const checker

Podrška za funkcije označene atributom `const` nije realizovana kroz poseban checker, već je objedinjena sa podrškom za `pure` funkcije u okviru klase `PureFunctionChecker`. Ovakav pristup je izabran zato što se veliki deo logike preklapa, uključujući prepoznavanje atributa, praćenje toka analize, beleženje sporednih efekata i prijavljivanje dijagnostičkih poruka. Time je izbegnuto dupliranje koda i pojednostavljeno dalje proširivanje implementacije.

Prilikom ulaska u funkciju proverava se da li je ona označena atributom `pure` ili `const`, nakon čega se u `ProgramState` beleži njen tip. Na osnovu ove informacije callback metode tokom daljeg toka analize primenjuju odgovarajuća pravila u zavisnosti od vrste funkcije koja se proverava.

Kod funkcija označenih atributom `const` proveravaju se isti sporedni efekti kao i kod `pure` funkcija, odnosno upis u globalne promenljive, upis preko pokazivača i upis preko referenci. Razlika u implementaciji odnosi se na obradu poziva drugih funkcija u okviru metode `checkPreCall()`.

Ukoliko pozivana funkcija ima dostupno telo, analiza se nastavlja njenim simboličkim izvršavanjem. Kada telo nije dostupno, odluka se donosi na osnovu atributa pozivane funkcije i tipa funkcije pozivaoca. U tom slučaju funkcija označena atributom `const` može da pozove isključivo drugu `const` funkciju. Poziv `pure` funkcije evidentira se kao `InsufficientlyPureCall`, dok se poziv funkcije bez odgovarajućeg atributa evidentira kao `UnknownCall`.

Na primer, u sledećem testu funkcija označena atributom `const` poziva funkciju označenu atributom `pure`:

Listing 2: Poziv funkcije koja je označena kao pure ali joj je nedostupno telo

```
1 [[clang::annotate("pure")]]  
2 int external_pure_function(int x);  
3  
4 [[clang::annotate("const")]]  
5 int const_function(int x)  
6 {  
7     return external_pure_function(x); // expected warning  
8 }
```

U ovom primeru telo funkcije `external_pure_function()` nije dostupno, pa analizator ne može da proveriti njenu implementaciju. Zbog toga se dozvoljenost poziva određuje na osnovu atributa. Pošto funkcija označena atributom `const` može da poziva samo drugu `const` funkciju, poziv `pure` funkcije beleži se kao `InsufficientlyPureCall`.

4 Zaključak

U okviru ovog rada implementiran je proširivi checker za Clang Static Analyzer namenjen proveriti funkcija označenih atributima `pure` i `const`. Implementacija je zasnovana na callback mehanizmu CSA i omogućava otkrivanje sporednih efekata koji narušavaju uslove definisane ovim atributima. Pored provere upisa u globalne promenljive, preko pokazivača i referenci, implementirana je i analiza poziva drugih funkcija uz razlikovanje slučajeva kada je telo pozivane funkcije dostupno i kada nije.

Podrška za oba atributa objedinjena je u okviru jednog checker-a, dok se odgovarajuća pravila primenjuju na osnovu tipa funkcije koja se trenutno analizira. Dodatno, omogućena je konfiguracija rada checker-a kroz više režima, čime korisnik može da odabere da li želi proveru samo `pure`, samo `const` ili obe vrste funkcija.

Implementirano rešenje potvrđeno je skupom test primera koji obuhvataju različite scenarije korišćenja atributa, uključujući ispravne i neispravne slučajeve. Dobijeni rezultati pokazuju da checker uspešno prepoznaje implementirane vrste sporednih efekata i generiše odgovarajuća upozorenja.

Dalji pravac razvoja može obuhvatiti podršku za složenije obrasce pristupa memoriji i precizniju obradu poziva funkcija čije telo nije dostupno.

Literatura

- [1] GNU Project. *Common Function Attributes*. GNU Compiler Collection (GCC). Sections “pure” and “const”, Accessed: 2026-07-26.